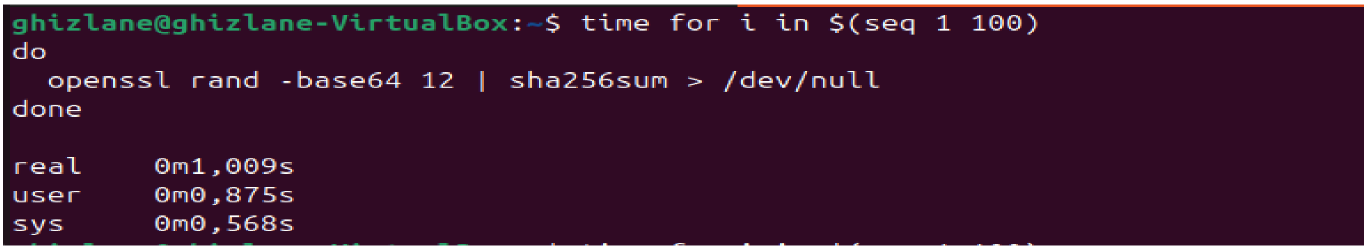
Rendu Introduction à la cybersécurité: KDF

Réalisée par Hassani ghizlane

Exercice 1:

1. Calculez le temps nécessaire pour générer le Hash de 100 empreintes de type sha256 issues de données aléatoires de 12 octets en base64,

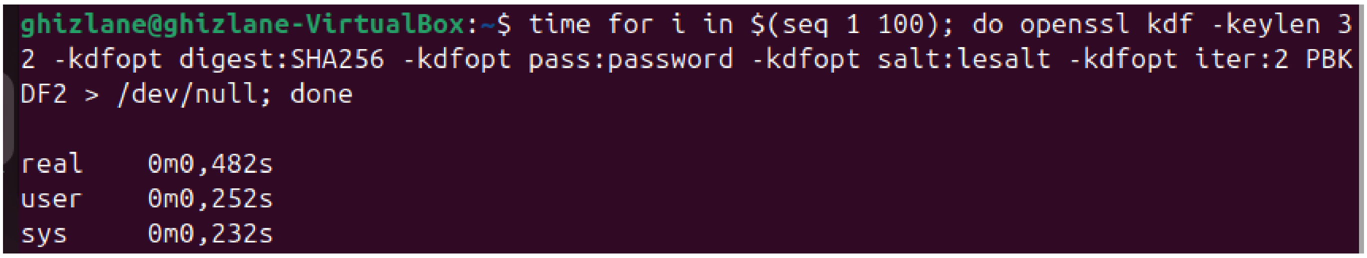
On génère une donnée aléatoire de 12 octets en base64, puis on calcule son empreinte avec SHA-256. Cette opération est très rapide, car SHA-256 est conçu pour produire un résultat en peu de temps. Il s'agit d'une simple fonction de hachage, faite pour produire une empreinte rapidement.



```
ghizlane@ghizlane-VirtualBox:~$ time for i in $(seq 1 100)
do
  openssl rand -base64 12 | sha256sum > /dev/null
done

real	0m1,009s
user	0m0,875s
sys	0m0,568s
```

2. Idem avec PBKDF2 avec 2 itérations,

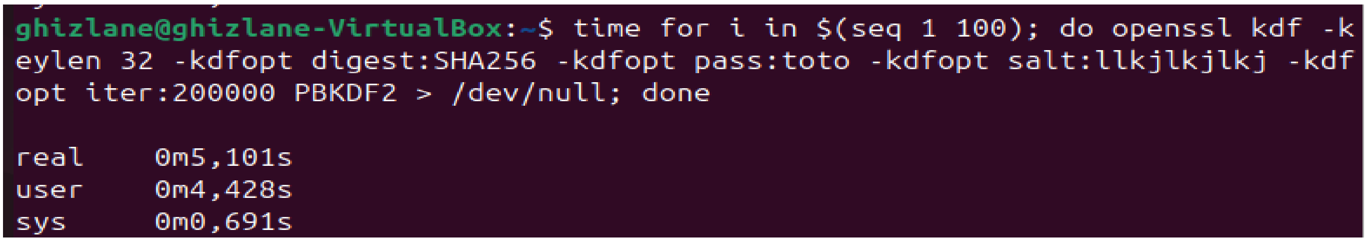
Ici, on utilise PBKDF2 avec seulement 2 itérations. Le temps de calcul reste faible, mais il est déjà un peu plus élevé qu'avec un simple SHA-256. Cela montre que PBKDF2 ajoute un coût de calcul supplémentaire.



```
ghizlane@ghizlane-VirtualBox:~$ time for i in $(seq 1 100); do openssl kdf -keylen 32 -kdfopt digest:SHA256 -kdfopt pass:password -kdfopt salt:lesalt -kdfopt iter:2 PBKDF2 > /dev/null; done

real	0m0,482s
user	0m0,252s
sys	0m0,232s
```

3. Idem avec PBKDF2 avec 200000 itérations,

Plus le nombre d'itérations augmente, plus la fonction répète le calcul, et plus le temps nécessaire devient important. C'est justement le but recherché. On ne cherche pas à aller vite, mais à rendre le calcul suffisamment lent pour décourager les attaques par force brute. Un attaquant qui voudrait tester des milliers ou des millions de mots de passe perdrait alors énormément de temps.



```
ghizlane@ghizlane-VirtualBox:~$ time for i in $(seq 1 100); do openssl kdf -keylen 32 -kdfopt digest:SHA256 -kdfopt pass:toto -kdfopt salt:llkjlkjlkj -kdfopt iter:200000 PBKDF2 > /dev/null; done

real	0m5,101s
user	0m4,428s
sys	0m0,691s
```

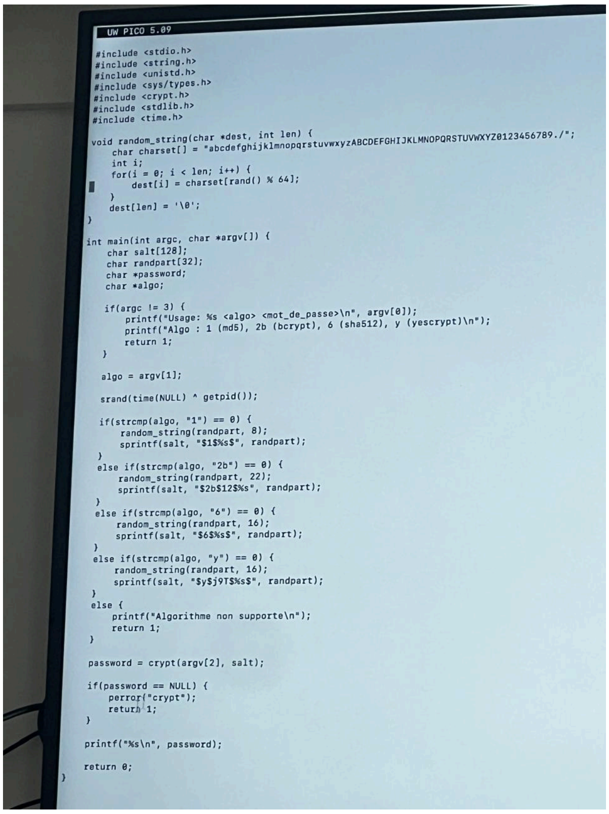
Interprétation :

On observe que le temps de calcul augmente avec le nombre d’itérations.
Le SHA-256 simple est le plus rapide, car il ne fait qu’un seul calcul.
PBKDF2 avec 2 itérations reste encore rapide, alors que PBKDF2 avec 200000 itérations devient beaucoup plus lent.
Cela montre que l’étirement de clé permet de ralentir volontairement le calcul pour mieux protéger les mots de passe.
→ Le cours présente justement PBKDF2 comme une fonction utilisant un mot de passe, un salt, une fonction de hachage et un nombre d’itérations pour produire une sortie dérivée.

Exercice 2:

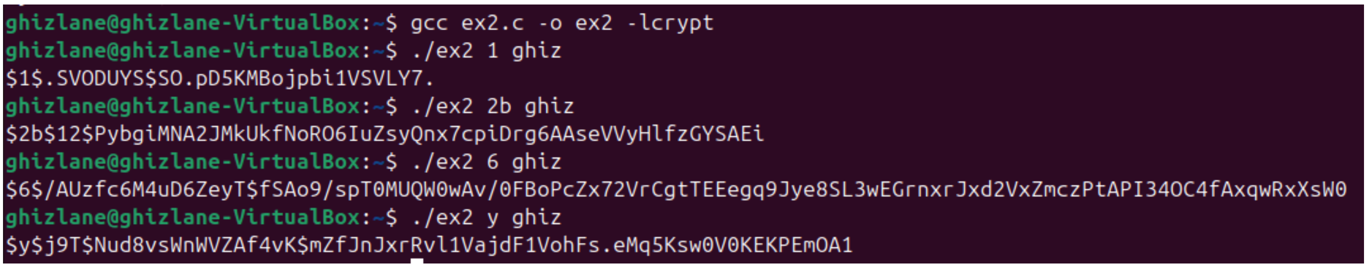
Dans le programme du cours, le mot de passe était saisi au clavier, puis la fonction `crypt()` était appelée avec un salt yescrypt déjà fixé dans le code.
Dans l’exercice, j’ai repris le même programme, j’ai supprimé la saisie au clavier pour que le mot de passe soit donné lors de l’exécution. Ensuite, il faut ajouter un premier argument, permettant de choisir l’algorithme à utiliser parmi md5, bcrypt, sha512 et yescrypt.
Enfin, au lieu d’utiliser un salt fixe, on génère automatiquement une partie aléatoire afin d’obtenir un hash différent à chaque exécution, même si le mot de passe reste le même.

J’ai en premier temps créer une fichier :
nano ex2.c
avec le code :



```
UW PICO 5.09

#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <crypt.h>
#include <stdlib.h>
#include <time.h>

void random_string(char *dest, int len) {
    char charset[] = "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789./";
    int i;
    for(i = 0; i < len; i++) {
        dest[i] = charset[rand() % 64];
    }
    dest[len] = '\0';
}

int main(int argc, char *argv[]) {
    char salt[128];
    char randpart[32];
    char *password;
    char *algo;

    if(argc != 3) {
        printf("Usage: %s <algo> <mot_de_passe>\n", argv[0]);
        printf("Algo : 1 (md5), 2b (bcrypt), 6 (sha512), y (yescrypt)\n");
        return 1;
    }

    algo = argv[1];

    srand(time(NULL) ^ getpid());

    if(strcmp(algo, "1") == 0) {
        random_string(randpart, 8);
        sprintf(salt, "$1$%s$", randpart);
    }
    else if(strcmp(algo, "2b") == 0) {
        random_string(randpart, 22);
        sprintf(salt, "$2b$12$%s", randpart);
    }
    else if(strcmp(algo, "6") == 0) {
        random_string(randpart, 16);
        sprintf(salt, "$6$%s$", randpart);
    }
    else if(strcmp(algo, "y") == 0) {
        random_string(randpart, 16);
        sprintf(salt, "$y$j9T$%s$", randpart);
    }
    else {
        printf("Algorithme non supporte\n");
        return 1;
    }

    password = crypt(argv[2], salt);

    if(password == NULL) {
        perror("crypt");
        return 1;
    }

    printf("%s\n", password);

    return 0;
}
```

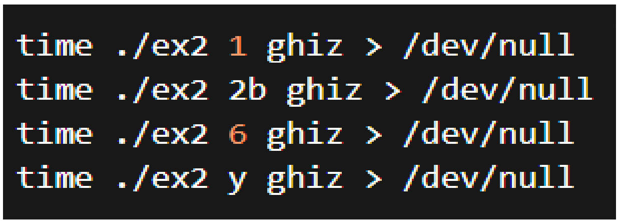
compilation : **gcc ex2.c -o ex2 -lcrypt**



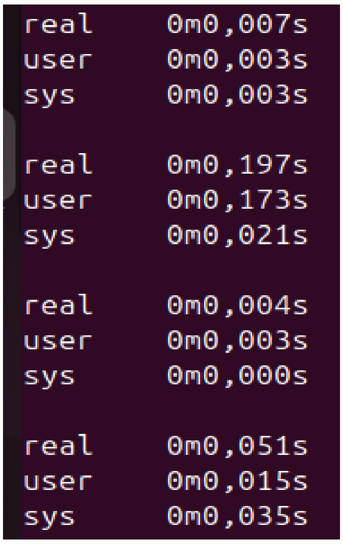
```
ghizlane@ghizlane-VirtualBox:~$ gcc ex2.c -o ex2 -lcrypt
ghizlane@ghizlane-VirtualBox:~$ ./ex2 1 ghiz
$1$.SVODUYS$SO.pD5KMBojpbi1VSVLY7.
ghizlane@ghizlane-VirtualBox:~$ ./ex2 2b ghiz
$2b$12$PybgiMNA2JMkUkfNoRO6IuZsyQnx7cpiDrg6AAseVVyHlfzGYSAEi
ghizlane@ghizlane-VirtualBox:~$ ./ex2 6 ghiz
$6$/AUzfc6M4uD6ZeyT$fSAo9/spT0MUQW0wAv/0FBoPcZx72VrCgtTEEegq9Jye8SL3wEGrnxrJxd2VxZmczPtAPI34OC4fAxqwRxXsW0
ghizlane@ghizlane-VirtualBox:~$ ./ex2 y ghiz
$y$j9T$Nud8vsWnWVZAf4vK$mZfJnJxrRvl1VajdF1VohFs.eMq5Ksw0V0KEKPEmOA1
```

Les différents tests montrent que le programme fonctionne correctement avec les quatre algorithmes demandés : md5, bcrypt, sha512 et yescrypt. Pour un même mot de passe, le hash obtenu change selon l'algorithme utilisé, car chacun possède son propre format. On remarque aussi que le résultat commence par un préfixe différent (`$1$`, `$2b$`, `$6$`, `$y$`).

Pour 1 mot de passe:

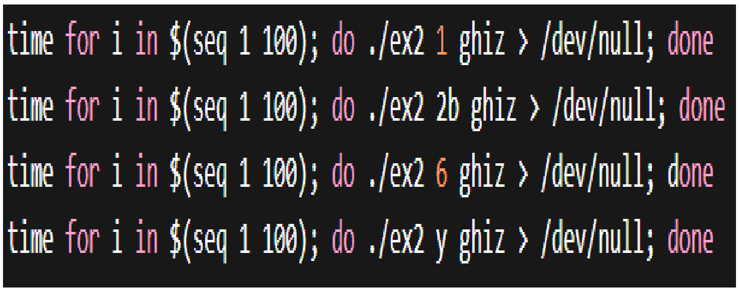


```
time ./ex2 1 ghiz > /dev/null
time ./ex2 2b ghiz > /dev/null
time ./ex2 6 ghiz > /dev/null
time ./ex2 y ghiz > /dev/null
```

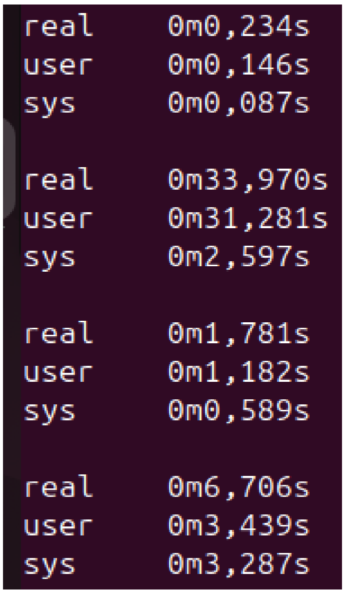


```
real	0m0,007s
user	0m0,003s
sys	0m0,003s

real	0m0,197s
user	0m0,173s
sys	0m0,021s

real	0m0,004s
user	0m0,003s
sys	0m0,000s

real	0m0,051s
user	0m0,015s
sys	0m0,035s
```

Pour 100 mot de passe:

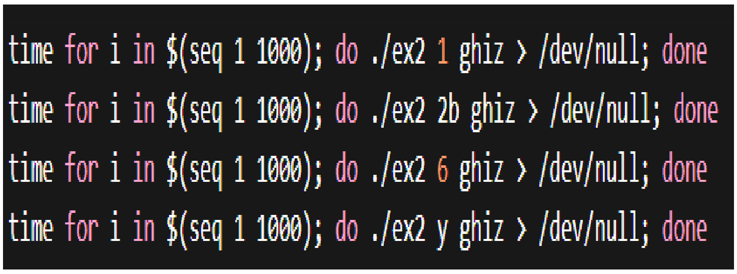


```
time for i in $(seq 1 100); do ./ex2 1 ghiz > /dev/null; done
time for i in $(seq 1 100); do ./ex2 2b ghiz > /dev/null; done
time for i in $(seq 1 100); do ./ex2 6 ghiz > /dev/null; done
time for i in $(seq 1 100); do ./ex2 y ghiz > /dev/null; done
```

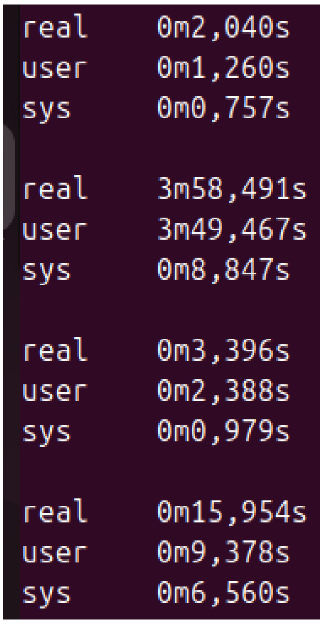


```
real	0m0,234s
user	0m0,146s
sys	0m0,087s

real	0m33,970s
user	0m31,281s
sys	0m2,597s

real	0m1,781s
user	0m1,182s
sys	0m0,589s

real	0m6,706s
user	0m3,439s
sys	0m3,287s
```

Pour 1000 mot de passe:



```
time for i in $(seq 1 1000); do ./ex2 1 ghiz > /dev/null; done
time for i in $(seq 1 1000); do ./ex2 2b ghiz > /dev/null; done
time for i in $(seq 1 1000); do ./ex2 6 ghiz > /dev/null; done
time for i in $(seq 1 1000); do ./ex2 y ghiz > /dev/null; done
```



```
real    0m2,040s
user    0m1,260s
sys     0m0,757s

real    3m58,491s
user    3m49,467s
sys     0m8,847s

real    0m3,396s
user    0m2,388s
sys     0m0,979s

real    0m15,954s
user    0m9,378s
sys     0m6,560s
```

Cet exercice montre donc l’importance du salt et l’intérêt d’utiliser des algorithmes adaptés au stockage des mots de passe, en particulier les algorithmes modernes comme bcrypt ou yescrypt, qui offrent une meilleure protection que les algorithmes plus anciens.